

Verified processes on the BEAM — graduated

The shipped API, the measured numbers, and the regression that proves it holds

graduated — priv modules shipped, tests green, numbers measured

2026-08-29

What this document adds

The companion document, *Verified processes on the BEAM*, made the case that a `receive` loop is a transition relation and walked the eight guarantees as prototypes. This one is the graduation record: the prototypes are now shipped `priv/` modules with a test suite, real timings, and a regression that proves the retrofit broke nothing. Where the first document argued, this one accounts.

FOUR MODULES, ONE MODEL

- `priv/footprint.bl` — an effect is a footprint `{resource → mode}`. The rung ladder is recovered as `rank(footprint)`, so it *is* the old effect model, generalized.
- `priv/model.bl` — the transition extractor: five surface forms lower to one graph, plus the naming trichotomy (binding · annotation · content-hash).
- `priv/system.bl` — the guarantee engine: inductive safety, abduction, deadlock, liveness, refinement, sandbox — each a function of a transition model plus the bundled `z3`.
- `priv/effects.bl` — retrofitted onto `footprint`, so the codebase now has *one* effect model, not two.

The shipped API

Every guarantee is a plain function; nothing is a special form. This is the whole thesis made concrete — you point these at ordinary code.

```
;; extraction (priv/model.bl)
(model/system-model node)      ; → {:name :transitions :graph}
(model/transitions node)      ; → [{:label :pattern :guard :next :body} ...]
(model/graph clauses)         ; → the (label, next) pairs — the machine's
shape
```

 Clojure

```
;; effects (priv/footprint.bl)
(fp/fp-walk env node)           ; → {resource → mode}
(fp/pure? fp) (fp/monotone? fp) ; → the algebra
(fp/commute? a b) (fp/frame-independent? a b)
(fp/rung fp)                     ; → :pure | :atom | :process | :io (the old
ladder)

;; guarantees (priv/system.bl)
(sys/prove-box port inv svar decls init transitions) ; □Inv, unbounded (z3)
(sys/abduce port inv svar decls guard next domain)  ; auto-strengthen
(sys/all-senders-guarantee? db q-fn label pred)      ; the sender discharge
(sys/deadlocked db q-fn)                             ; datalog cycle query
(sys/find-lasso step worst progress phase-of init k) ; BMC liveness
(sys/simulates? port R decls a-var c-var a-next c-next extra) ; refinement
(sys/migration-preserves? port inv decls v1 v2 migrate) ; hot upgrade
(sys/footprint-in-caps? fp caps) (sys/spawn-footprint env node) ; sandbox
```

The assurances are tests

The guarantees graduate as `deftest` assertions in the suite, not just demos — so a regression that broke one would fail the build.

```
$ BeamLisp.TestRT.cli(["test/bl/system/model_test.bl", "test/bl/system/
system_test.bl"])
```

```
system.model-test    4 tests / 10 assertions  - 0 failures
system.system-test   15 tests / 30 assertions  - 0 failures
```

What those forty assertions pin down, by guarantee:

test	the assurance it holds
five-forms-one-graph	<code>defserver ≡ loop ≡ defmulti</code> — the same graph, exactly
guard-reported-faithfully	<code>:when</code> captured, never a fabricated guard
naming-trichotomy	binding name · content-hash, stable across identical structure
inductive-invariant-proven-unbounded	<code>□(balance ≥ 0)</code> for every state and amount
unguarded-withdraw-caught-with-witness	the failing transition named, z3 counterexample
abduction-finds-sound-precondition	enumerate + decide → a genuinely sufficient <code>P</code>

abduction-honest-when-vocabulary-too-small	<code>[]</code> over too small a domain, no false proof
sender-discharge-only-we-can-do	all callers guarantee <code>P</code> $\rightarrow$ no annotation; one careless caller found
sync-call-cycle-is-a-named-deadlock	the three cycle members named, acyclic excluded
buggy-handler-lasso-found-with-witness	the <code>:retry</code> spin, with its trace
liveness-honesty-is-structural	approximate vs exact is a query
verified-hot-upgrade	migration preserves <code>Inv</code> , <code>v2</code> simulates <code>v1</code>
broken-migration-violates-simulation	a zeroing migration breaks <code>R(v1,v2)</code>
sandbox-containment-and-fleet-noninterference	<code>footprint $\subseteq$ caps</code> , disjoint workers commute
unconveyed-spawn-escalates	un-conveyed spawn $\rightarrow$ <code>:opaque-world</code> ; conveyed stays bounded

The regression: the retrofit broke nothing

The riskiest move was folding `effects.bl` onto `footprint` — `effects` has a live consumer (`priv/live.bl`, MVP-F). It is byte-identical after the retrofit, and the whole non-auth suite stays green.

```
$ BeamLisp.TestRT.cli(non-auth test/bl/**/*bl)
```

```
Ran 626 tests containing 1705 assertions.
0 failures, 0 errors.
```

The effect retrofit is output-identical: the same per-function effects and the same purity warning as before the change, and demo 06 (the `effects` consumer) still passes. The old `effect-seeds` table — a parallel copy of `footprint`'s classification — is deleted. One effect model now, not two.

auth Pre-existing, out of scope: 64 auth errors

The full suite also reports 64 errors in `auth.*` tests (`undefined var: auth.biscuit.datalog/encode-block*`, though it is defined at `datalog.bl:528`). These reproduce in *isolation* with none of the graduation changes loaded, and the auth commits are ancestors of the merge base — a foreign-session load-order issue in the biscuit module, untouched by this work. Reported honestly rather than hidden; not a graduation regression.

The measured numbers

Warm, on the bundled z3 (`priv/z3/bin/z3` , 4.16.0), via the direct harness.

operation	time
footprint of one form (no solver)	0.08 ms
extract a <code>defserver</code> $\rightarrow$ <code>SystemModel</code>	21 ms
inductive <code>prove-box</code> (2 transitions, z3)	12 ms
refinement <code>simulates?</code> (one z3 query)	3.6 ms
<code>abduce</code> (400 candidates, z3 per candidate)	111 ms

WHY THESE NUMBERS ARE THE RIGHT SHAPE

The arithmetic guarantees cost roughly $(\text{number of transitions}) \times (\text{one z3 query at } 2\text{--}6 \text{ ms})$. They scale with the *size of the program*, not the size of the state space — which is the entire point of proving `□Inv` by induction instead of exploring states. `abduce` is the outlier at 111 ms because it is 400 candidate structures each decided by z3; it runs once, when an invariant needs strengthening, not on every check. The structural guarantees — footprint, deadlock — touch no solver at all and are sub-millisecond to a few milliseconds over the datalog fixpoint.

The pressure test, as a transcript

The demos in `examples/system/` are the pressure test made re-runnable. The load-bearing one — an OTP `defserver` , written with no verification in mind, produces the same machine as a raw `receive` loop:

```
$ mix beam_lisp.run --path priv examples/system/01_extraction.bl
```

```
raw receive-loop:  ([:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"])
defserver (OTP):   ([:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"])
defmulti dispatch: ([:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"])
all three identical: true
```

And the crown, end to end, on the graduated engine:

```
$ mix beam_lisp.run --path priv examples/system/08_hot_upgrade.bl
```

```
VERIFIED HOT UPGRADE — code_change from v1 {:balance} to v2 {:balance :currency}
1. migration b2 := b1 preserves balance ≥ 0:           true
2. v2.withdraw simulates v1.withdraw (observable identical): true
3. a migration that zeroes the balance is caught:       true
```

Graduation refinements applied

Two rough edges the prototype README flagged, now fixed in the shipped modules:

Refinement 1 — `footprint` carries a `pure-ops` set; `str`, `+`, `count`, `get`, and friends contribute the empty footprint. `(+ (deref a) (count xs))` is now `{a :R}`, clean.

Because — pure arithmetic touches no resource, so it must contribute no footprint entry — naming `:opaque-world` for `(+ 1 2)` was noise that weakened every downstream `pure?` and `commute?` read

Refinement 2 — `system/spawn-footprint` marks an un-conveyed `(erlang/spawn (fn ...))` as `{:opaque-world :X}` (escaping every finite cap set), while a conveyed `(erlang/spawn (bind env (fn ...)))` contributes only its bounded body footprint.

Because — the security-relevant fact about a spawn is whether the child carries the granting environment; an un-conveyed child runs at full authority, which the naive first-argument model missed

The seam: point-and-verify (P17)

The graduation above left the tier as libraries you hand-fed: every test wrote the transition maps and the SMT strings by hand. P17 closes that seam. The modules are now one package (`priv/system/`, namespace `system.*`), and one function does the whole job.

ANNOTATE A SERVER, IT GETS VERIFIED

```
(defserver ^{:invariant (>= balance 0)} account
  (init [balance] (ok balance))
  (handle-call [:deposit amt] [_ balance] :when (>= amt 0)
    (reply :ok (+ balance amt)))
  (handle-call [:withdraw amt] [_ balance] :when (and (>= amt 0) (>= balance amt))
    (reply :ok (- balance amt))))

(system/verify-process port (read account))
;; → {:name "account" :checked true :holds true :warnings ()}
```

The only thing that makes this ordinary server verifiable is the `^{:invariant}` on its name. `verify-process` extracts the transitions, reads the annotation (metadata is data), translates the guards to SMT (`system.smt`, which desugars `inc` $\rightarrow$ `+1`, `pos?` $\rightarrow$ `>0`, and flags anything it cannot model rather than emitting a false proof), proves $\Box \text{Inv}$, and returns the verdict.

Remove the withdraw guard and the checker renders the failure exactly like every other beam-lisp warning — `file:line:col`, the source line, a caret:

```
$ mix beam_lisp.run --path priv examples/system/09_point_and_verify.bl
```

```
account.bl:1:1: invariant not preserved by withdraw
1 | (defserver ^{:invariant (>= balance 0)} account
  | ^
```

And the two structural guarantees the earlier scorecard listed but had not built are now real. Message coverage is the headline one — the check a specification without an implementation structurally cannot make:

```
$ examples/system/09_point_and_verify.bl
```

```
the server handles: #{:deposit :withdraw}
a caller sends :deposit, :withdraw, and :close ...
unhandled (latent FunctionClauseError): (:close)
```

A message sent to the server that it does not handle is a latent `FunctionClauseError`, found from the whole call graph by a join of the senders (`:call/*` datoms) against the handled labels. Dispatch determinism is its `z3` companion: two same-label clauses whose guards can hold at once are a nondeterministic dispatch, and `z3` finds the overlap. Both ship in `system.core`, both tested.

The state graph also graduated to the repo's own relation algebra: `:~step` is a `defrelation` (a computed relation registered in the `datom` catalog), so reachability is a fixpoint over it and the counterexample is a backward path query — the model as a queryable view, not a bespoke search.

Where it stands

The system-model tier is graduated and integrated: one package (`priv/system/`, five modules under `system.*`), 65 test assertions green across model, guarantee engine, and seam, ten runnable demos, one unified effect model, a point-and-verify checker that reads `^{:invariant}` off an ordinary server, and the measured numbers above. Everything runs on the bundled `z3` with no JVM. The one honest debt is external — the pre-existing `auth.*` breakage — and it is out of this tier's scope. What remains is depth, not gaps: richer state models (beyond single-integer state) for `verify-process`, and wiring `:~step` into the full query planner so reachability is a plain `datom/q` rather than a helper.

Modules: `priv/{footprint,model,system,effects}.bl`. Tests: `test/bl/system/{model,system}_test.bl`. Demos: `examples/system/01..08`. Numbers + regression: `research/pl6_model/GRADUATION.md`. Plan: `!tasks/plans/PLAN-048`. Companion: `docs/typing/verified-processes.typ`.